Minitest Setup

One of the clearest differences between the RSpec way and the Minitest way is how you set up different data for different tests. In RSpec, the describe method is used to enclose an arbitrary number of tests for the purpose of giving each group of tests a different setup. For example, your Project specs in spec/models/project_spec.rb has three different groups, one without a task, one with a task, and one with a bunch of tasks.

In Minitest, each class has one setup method. (Technically, you can define multiple setup do blocks, but all of them are executed for all tests—you can't use Minitest setup to have different setups for different tests.) While you could solve this problem by putting each setup in a different class, I think it's somewhat more common to have explicit methods called from the beginning of the test to handle setup.

Here's one way to handle multiple setup blocks in a Minitest version of your Project tests:

minitest/01/test/models/project_test.rb

```
require "test_helper"

class ProjectTest < ActiveSupport::TestCase
  setup do
    @project = FactoryBot.create(:project)
  end

  def add_one_task
    @task = FactoryBot.create(:task, project: @project)
  end

  def add_many_tasks
    newly_done = FactoryBot.build(:task, :newly_complete)
    old_done = FactoryBot.build(:task, :long_complete, :small)
    small_not_done = FactoryBot.build(:task, :small)
    large_not_done = FactoryBot.build(:task, :large)
    @project.tasks = [newly_done, old_done, small_not_done, large_not_done]
    @project.save!
  end

  test "a project with no tasks is done" do
    assert(@project.done?)
  end

  test "a project with no tests is estimated" do
    assert_equal(0, @project.completed_velocity)
    assert_equal(0, @project.current_rate)
    assert(@project.projected_days_remaining.nan?)
    refute(@project.on_schedule?)
  end
```